Código fuente: Carga dinámica

Balanceo de carga estático vs. dinámico (Maestro–Trabajador) con Python + mpi4py

Jared Méndez Valencia · Daniel del Valle Andrade · Johan Lezama Núñez · Camila Serrano Trinidad

Contents

1 balanceo.py 1

2 experimentos.py 5

3 graficas.py 7

4 tabla_latex.py 12

5 cluster_diagnostico.py 13

6 servidor_demo.py 15

7 cluster/hosts.txt.ejemplo 17

1 balanceo.py

Programa principal (mpi4py): maestro, trabajadores y estrategias de reparto.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
balanceo.py - Balanceo de carga ESTÁTICO vs DINÁMICO con arquitectura
Maestro-Trabajador, usando mpi4py.

Idea: un "nodo" es un proceso MPI. Como en una laptop todos los procesos
tienen la misma velocidad, se SIMULAN nodos lentos y rápidos: cada trabajador
tiene un factor de velocidad (1.0 = rápido, 0.5 = mitad de velocidad, ...) y
tarda  costo / velocidad  en ejecutar una tarea.

    rank 0      -> MAESTRO   (reparte paquetes de trabajo, no calcula)
    rank 1..n-1 -> TRABAJADORES (los "nodos" heterogéneos)

Estrategias (--estrategia):
    estatico            bloques contiguos del mismo tamaño (decidido de antemano)
    estatico_ciclico    tarea i -> trabajador i % W (round-robin, de antemano)
    estatico_ponderado  bloques proporcionales a la velocidad CONOCIDA de cada nodo
    dinamico            el trabajador pide un paquete de --chunk tareas cuando se desocupa
    guiado              dinámico con paquetes que se encogen: restantes / (factor * W)

Ejemplo:
    mpiexec -n 5 python balanceo.py --estrategia dinamico --chunk 4 \
            --velocidades 1,1,0.5,0.25
"""
import argparse
import json
import math
import sys
import time

import numpy as np
from mpi4py import MPI

# Etiquetas (tags) de los mensajes MPI
TAG_PEDIR = 1     # trabajador -> maestro : "estoy libre, dame trabajo"
TAG_TRABAJO = 2   # maestro -> trabajador : paquete de tareas
TAG_FIN = 3       # maestro -> trabajador : ya no hay más trabajo

ESTRATEGIAS = ["estatico", "estatico_ciclico", "estatico_ponderado",
               "dinamico", "guiado"]


# ---------------------------------------------------------------------------
# Generación de la carga de trabajo
# ---------------------------------------------------------------------------
def generar_costos(n, costo_medio_ms, variabilidad, semilla):
    """Costo (en segundos, a velocidad 1.0) de cada tarea.

    variabilidad = 0   -> todas las tareas cuestan lo mismo.
    variabilidad > 0   -> distribución log-normal (algunas tareas son mucho
                          más pesadas), con media = costo_medio_ms.
    """
    rng = np.random.default_rng(semilla)
    if variabilidad <= 0:
        costos = np.full(n, costo_medio_ms / 1000.0)
    else:
        mu = math.log(costo_medio_ms) - variabilidad ** 2 / 2.0
```

```
        costos = rng.lognormal(mu, variabilidad, n) / 1000.0
    return np.clip(costos, 0.0005, None)


# ---------------------------------------------------------------------------
# Ejecución de una tarea en un nodo con cierta velocidad
# ---------------------------------------------------------------------------
def contar_primos(cantidad):
    """Carga CPU real: cuenta primos entre 10^6 y 10^6 + cantidad."""
    total = 0
    for x in range(1_000_001, 1_000_001 + cantidad, 2):
        es_primo = True
        i = 3
        while i * i <= x:
            if x % i == 0:
                es_primo = False
                break
            i += 2
        total += es_primo
    return total


def calibrar_real():
    """Cuántos números por milisegundo procesa contar_primos en esta máquina."""
    t = time.perf_counter()
    contar_primos(4000)
    return 4000 / max((time.perf_counter() - t) * 1000.0, 1e-6)


def ejecutar_tarea(costo, velocidad, modo, num_por_ms):
    if modo == "simulado":
        # el nodo "tarda" costo/velocidad (dormir = no estorba a los demás)
        time.sleep(costo / velocidad)
    else:
        # modo real: trabajo CPU de verdad; un nodo con velocidad 1/k
        # repite el trabajo k veces (0.5 -> 2 veces, 0.25 -> 4 veces)
        repeticiones = max(1, round(1.0 / velocidad))
        for _ in range(repeticiones):
            contar_primos(int(costo * 1000.0 * num_por_ms))


# ---------------------------------------------------------------------------
# Reparto ESTÁTICO (se decide antes de empezar, sin mirar cómo avanza todo)
# ---------------------------------------------------------------------------
def reparto_estatico(costos, n_trab, estrategia, velocidades):
    n = len(costos)
    paquetes = [[] for _ in range(n_trab)]
    if estrategia == "estatico":
        base, extra = divmod(n, n_trab)
        ini = 0
        for w in range(n_trab):
            tam = base + (1 if w < extra else 0)
            paquetes[w] = costos[ini:ini + tam].tolist()
            ini += tam
    elif estrategia == "estatico_ciclico":
        for i, c in enumerate(costos):
            paquetes[i % n_trab].append(float(c))
    elif estrategia == "estatico_ponderado":
        v = np.array([velocidades[w % len(velocidades)] for w in range(n_trab)])
        cuota = n * v / v.sum()
        cuentas = np.floor(cuota).astype(int)
        resto = n - cuentas.sum()
        for w in np.argsort(-(cuota - cuentas))[:resto]:
            cuentas[w] += 1
        ini = 0
        for w in range(n_trab):
            paquetes[w] = costos[ini:ini + cuentas[w]].tolist()
            ini += cuentas[w]
    return paquetes


# ---------------------------------------------------------------------------
# Tamaño del paquete en el reparto DINÁMICO
# ---------------------------------------------------------------------------
def tam_paquete(estrategia, restantes, n_trab, chunk, factor):
    if estrategia == "dinamico":
        return min(chunk, restantes)
    # guiado: paquetes grandes al inicio, pequeños al final (menos esperas)
    return min(restantes, max(chunk, math.ceil(restantes / (factor * n_trab))))


# ---------------------------------------------------------------------------
# MAESTRO
# ---------------------------------------------------------------------------
def maestro(comm, args, costos):
    n_trab = comm.Get_size() - 1
    info = {"paquetes": 0, "mensajes": 0}

    if args.estrategia.startswith("estatico"):
        # Un solo envío por trabajador, decidido de antemano (Scatter)
        velocidades = [float(v) for v in args.velocidades.split(",")]
        paquetes = reparto_estatico(costos, n_trab, args.estrategia, velocidades)
        comm.scatter([[]] + paquetes, root=0)
```

```
        info["paquetes"] = n_trab
        info["mensajes"] = n_trab
    else:
        # Cola de trabajo: el que se desocupa pide más (pull / self-scheduling)
        st = MPI.Status()
        siguiente, n, activos = 0, len(costos), n_trab
        while activos > 0:
            comm.recv(source=MPI.ANY_SOURCE, tag=TAG_PEDIR, status=st)
            w = st.Get_source()
            info["mensajes"] += 1
            if siguiente < n:
                tam = tam_paquete(args.estrategia, n - siguiente, n_trab,
                                  args.chunk, args.factor_guiado)
                comm.send(costos[siguiente:siguiente + tam].tolist(),
                          dest=w, tag=TAG_TRABAJO)
                siguiente += tam
                info["paquetes"] += 1
            else:
                comm.send(None, dest=w, tag=TAG_FIN)
                activos -= 1
            info["mensajes"] += 1
    return info


# ---------------------------------------------------------------------------
# TRABAJADOR
# ---------------------------------------------------------------------------
def trabajador(comm, args, t0, num_por_ms):
    rank = comm.Get_rank()
    lista_v = [float(v) for v in args.velocidades.split(",")]
    vel_base = lista_v[(rank - 1) % len(lista_v)]

    # --degradar "rank:segundos:nueva_velocidad"  (un nodo se vuelve lento
    # a mitad de la ejecución, p. ej. otro proceso le roba CPU)
    deg = None
    if args.degradar:
        r, t, v = args.degradar.split(":")
        if int(r) == rank:
            deg = (float(t), float(v))

    def velocidad_ahora():
        if deg and (time.perf_counter() - t0) >= deg[0]:
            return deg[1]
        return vel_base

    intervalos = []          # (inicio, fin, num. tareas) relativo a t0
    espera, tareas = 0.0, 0

    def procesar(lista_costos):
        nonlocal tareas
        ini = time.perf_counter()
        for c in lista_costos:
            ejecutar_tarea(c, velocidad_ahora(), args.modo, num_por_ms)
        fin = time.perf_counter()
        tareas += len(lista_costos)
        intervalos.append((ini - t0, fin - t0, len(lista_costos)))

    if args.estrategia.startswith("estatico"):
        mi_paquete = comm.scatter(None, root=0)
        procesar(mi_paquete)
    else:
        st = MPI.Status()
        while True:
            t_a = time.perf_counter()
            comm.send(None, dest=0, tag=TAG_PEDIR)
            if args.latencia_ms > 0:                    # latencia de red simulada
                time.sleep(args.latencia_ms / 1000.0)
            paquete = comm.recv(source=0, tag=MPI.ANY_TAG, status=st)
            espera += time.perf_counter() - t_a
            if st.Get_tag() == TAG_FIN:
                break
            procesar(paquete)

    ocupado = sum(f - i for i, f, _ in intervalos)
    return {"rank": rank, "velocidad": vel_base, "tareas": tareas,
            "ocupado": ocupado, "espera_maestro": espera,
            "intervalos": intervalos}


# ---------------------------------------------------------------------------
# Programa principal
# ---------------------------------------------------------------------------
def leer_argumentos():
    p = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--estrategia", choices=ESTRATEGIAS, default="dinamico")
    p.add_argument("--tareas", type=int, default=240, help="número total de tareas")
    p.add_argument("--costo-medio-ms", type=float, default=10.0)
    p.add_argument("--variabilidad", type=float, default=0.8,
                   help="sigma log-normal del costo (0 = tareas idénticas)")
    p.add_argument("--semilla", type=int, default=42)
    p.add_argument("--velocidades", default="1,1,0.5,0.25",
                   help="velocidad de cada trabajador (se repite en ciclo)")
```

```
    p.add_argument("--chunk", type=int, default=4,
                   help="tareas por paquete (dinámico) o paquete mínimo (guiado)")
    p.add_argument("--factor-guiado", type=float, default=2.0)
    p.add_argument("--latencia-ms", type=float, default=0.0,
                   help="latencia de red simulada por petición al maestro")
    p.add_argument("--modo", choices=["simulado", "real"], default="simulado")
    p.add_argument("--degradar", default="",
                   help='"rank:segundos:nueva_velocidad"')
    p.add_argument("--salida", default="", help="archivo .json para guardar resultados")
    p.add_argument("--guardar-intervalos", action="store_true")
    return p.parse_args()


def main():
    args = leer_argumentos()
    comm = MPI.COMM_WORLD
    rank, size = comm.Get_rank(), comm.Get_size()
    if size < 2:
        if rank == 0:
            sys.exit("Se necesitan al menos 2 procesos (1 maestro + 1 trabajador): "
                     "mpiexec -n 5 python balanceo.py ...")
        return
    n_trab = size - 1

    costos = None
    num_por_ms = 1.0
    if rank == 0:
        costos = generar_costos(args.tareas, args.costo_medio_ms,
                                args.variabilidad, args.semilla)
        if args.modo == "real":
            num_por_ms = calibrar_real()
    num_por_ms = comm.bcast(num_por_ms, root=0)

    comm.Barrier()
    t0 = time.perf_counter()
    if rank == 0:
        info = maestro(comm, args, costos)
        mi = None
    else:
        mi = trabajador(comm, args, t0, num_por_ms)
    comm.Barrier()
    makespan = time.perf_counter() - t0
    todos = comm.gather(mi, root=0)

    if rank != 0:
        return

    # ---------------------------- métricas ------------------------------
    trab = [t for t in todos if t is not None]
    total_trabajo = float(costos.sum())
    vel = [t["velocidad"] for t in trab]
    t_ideal = total_trabajo / sum(vel)              # balance perfecto, sin costos extra
    t_1_nodo = total_trabajo / max(vel)             # el nodo más rápido trabajando solo
    for t in trab:
        t["ocioso"] = makespan - t["ocupado"]
        t["utilizacion"] = t["ocupado"] / makespan
    ocupados = [t["ocupado"] for t in trab]
    res = {
        "estrategia": args.estrategia, "chunk": args.chunk,
        "trabajadores": n_trab, "tareas": args.tareas,
        "velocidades": vel, "latencia_ms": args.latencia_ms,
        "modo": args.modo, "degradar": args.degradar,
        "makespan": makespan, "t_ideal": t_ideal, "t_1_nodo": t_1_nodo,
        "speedup": t_1_nodo / makespan,
        "eficiencia_balanceo": t_ideal / makespan,
        "utilizacion_media": float(np.mean([t["utilizacion"] for t in trab])),
        "ocioso_total": float(sum(t["ocioso"] for t in trab)),
        "desbalance": max(ocupados) / (sum(ocupados) / len(ocupados)),
        "paquetes": info["paquetes"], "mensajes": info["mensajes"],
        "trabajadores_detalle": trab if args.guardar_intervalos else
            [{k: v for k, v in t.items() if k != "intervalos"} for t in trab],
    }

    etiqueta = args.estrategia + (f" (chunk={args.chunk})"
                                  if args.estrategia in ("dinamico", "guiado") else "")
    print(f"\n=== {etiqueta} | {n_trab} trabajadores | {args.tareas} tareas "
          f"| modo {args.modo} ===")
    print(f"Tiempo total (makespan): {makespan:.3f} s   "
          f"ideal: {t_ideal:.3f} s   1 nodo rápido: {t_1_nodo:.3f} s")
    print(f"Speedup: {res['speedup']:.2f}x   eficiencia de balanceo: "
          f"{100 * res['eficiencia_balanceo']:.1f} %   "
          f"utilización media: {100 * res['utilizacion_media']:.1f} %")
    print(f"Paquetes: {info['paquetes']}   mensajes: {info['mensajes']}")
    print(f"{'rank':>4} {'vel':>5} {'tareas':>7} {'ocupado':>9} {'ocioso':>8} {'util':>6}")
    for t in trab:
        print(f"{t['rank']:>4} {t['velocidad']:>5.2f} {t['tareas']:>7} "
              f"{t['ocupado']:>8.3f}s {t['ocioso']:>7.3f}s "
              f"{100 * t['utilizacion']:>5.1f}%")

    if args.salida:
        with open(args.salida, "w", encoding="utf-8") as f:
            json.dump(res, f, ensure_ascii=False, indent=1)

```

```
338
339 if __name__ == "__main__":
340     main()
```

2 experimentos.py

Ejecuta los escenarios y guarda los resultados en JSON.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
experimentos.py - Corre balanceo.py con distintos escenarios y guarda
los resultados (mediana de varias repeticiones) en resultados/*.json.

    python experimentos.py                # todos los experimentos
    python experimentos.py --exp chunk    # solo uno
    python experimentos.py --reps 5       # más repeticiones (menos ruido)

Escenarios:
    homogeneo      4 nodos iguales, tareas de costo variable
    heterogeneo    nodos con velocidades 1, 1, 0.5, 0.25
    chunk          tamaño del paquete vs. latencia de red (compromiso)
    escalabilidad  1, 2, 4 y 8 trabajadores
    degradado      un nodo se vuelve lento a mitad de la ejecución
    gantt          una corrida guardando intervalos (línea de tiempo)
"""
import argparse
import json
import os
import shutil
import statistics
import subprocess
import sys
import tempfile

AQUI = os.path.dirname(os.path.abspath(__file__))
SALIDA = os.path.join(AQUI, "resultados")
HETERO = "1,1,0.5,0.25"


def lanzador():
    exe = shutil.which("mpiexec") or shutil.which("mpirun")
    if not exe:
        sys.exit("No encuentro mpiexec/mpirun. Instala MPICH u OpenMPI (o MS-MPI en Windows).")
    return exe


def correr(estrategia, trabajadores, velocidades, reps, extra=(), chunk=1):
    """Ejecuta una configuración `reps` veces y devuelve la corrida mediana."""
    corridas = []
    for r in range(reps):
        with tempfile.NamedTemporaryFile(suffix=".json", delete=False) as tmp:
            ruta = tmp.name
        cmd = [lanzador(), "-n", str(trabajadores + 1), sys.executable,
               os.path.join(AQUI, "balanceo.py"),
               "--estrategia", estrategia, "--chunk", str(chunk),
               "--velocidades", velocidades, "--salida", ruta, *extra]
        subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL)
        with open(ruta, encoding="utf-8") as f:
            corridas.append(json.load(f))
        os.remove(ruta)
    corridas.sort(key=lambda c: c["makespan"])
    mediana = corridas[len(corridas) // 2]
    mediana["makespans"] = [c["makespan"] for c in corridas]
    mediana["makespan_mediana"] = statistics.median(mediana["makespans"])
    return mediana


def guardar(nombre, datos):
    os.makedirs(SALIDA, exist_ok=True)
    with open(os.path.join(SALIDA, nombre + ".json"), "w", encoding="utf-8") as f:
        json.dump(datos, f, ensure_ascii=False, indent=1)
    print(f"  -> resultados/{nombre}.json")


def etiqueta(est, chunk):
    return f"{est} (chunk={chunk})" if est in ("dinamico", "guiado") else est


def exp_comparacion(nombre, velocidades, reps, extra=(), incluir_ponderado=False):
    print(f"[{nombre}] velocidades={velocidades}")
    configs = [("estatico", 1), ("estatico_ciclico", 1)]
    if incluir_ponderado:
        configs.append(("estatico_ponderado", 1))
    configs += [("dinamico", 1), ("dinamico", 4), ("guiado", 1)]
    n_trab = len(velocidades.split(","))
    datos = []
    for est, chunk in configs:
        d = correr(est, n_trab, velocidades, reps, extra, chunk)
        print(f"  {etiqueta(est, chunk):28s} {d['makespan_mediana']:.3f} s")
        datos.append(d)
    guardar(nombre, datos)


def exp_chunk(reps):
    print("[chunk] latencia de red simulada = 3 ms por petición")
    extra = ("--latencia-ms", "3")
```

```
    datos = []
    for chunk in (1, 2, 4, 8, 16, 32, 60):
        d = correr("dinamico", 4, HETERO, reps, extra, chunk)
        print(f"  dinamico chunk={chunk:<3d} {d['makespan_mediana']:.3f} s  "
              f"paquetes={d['paquetes']}")
        datos.append(d)
    d = correr("guiado", 4, HETERO, reps, extra, 1)
    print(f"  guiado               {d['makespan_mediana']:.3f} s  paquetes={d['paquetes']}")
    datos.append(d)
    d = correr("estatico_ponderado", 4, HETERO, reps, extra, 1)
    print(f"  estatico_ponderado   {d['makespan_mediana']:.3f} s")
    datos.append(d)
    guardar("chunk", datos)


def exp_escalabilidad(reps):
    print("[escalabilidad] 480 tareas, velocidades en ciclo 1,1,0.5,0.25")
    datos = []
    for w in (1, 2, 4, 8):
        for est, chunk in (("estatico", 1), ("dinamico", 4), ("guiado", 1)):
            d = correr(est, w, HETERO, reps, ("--tareas", "480"), chunk)
            print(f"  W={w} {etiqueta(est, chunk):22s} {d['makespan_mediana']:.3f} s  "
                  f"speedup={d['t_1_nodo'] / d['makespan_mediana']:.2f}")
            datos.append(d)
    guardar("escalabilidad", datos)


def exp_degradado(reps):
    print("[degradado] 4 nodos iguales; el rank 4 baja a velocidad 0.25 a los 0.3 s")
    extra = ("--degradar", "4:0.3:0.25")
    datos = []
    for est, chunk in (("estatico", 1), ("estatico_ciclico", 1),
                       ("dinamico", 4), ("guiado", 1)):
        d = correr(est, 4, "1,1,1,1", reps, extra, chunk)
        print(f"  {etiqueta(est, chunk):28s} {d['makespan_mediana']:.3f} s")
        datos.append(d)
    guardar("degradado", datos)


def exp_gantt():
    print("[gantt] corridas con intervalos guardados (heterogéneo)")
    for est, chunk in (("estatico", 1), ("dinamico", 4), ("guiado", 1)):
        d = correr(est, 4, HETERO, 1, ("--guardar-intervalos",), chunk)
        guardar(f"gantt_{est}", d)


def main():
    p = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--exp", default="todos",
                   choices=["todos", "homogeneo", "heterogeneo", "chunk",
                            "escalabilidad", "degradado", "gantt"])
    p.add_argument("--reps", type=int, default=3)
    a = p.parse_args()
    t = a.exp
    if t in ("todos", "homogeneo"):
        exp_comparacion("homogeneo", "1,1,1,1", a.reps)
    if t in ("todos", "heterogeneo"):
        exp_comparacion("heterogeneo", HETERO, a.reps, incluir_ponderado=True)
    if t in ("todos", "chunk"):
        exp_chunk(a.reps)
    if t in ("todos", "escalabilidad"):
        exp_escalabilidad(a.reps)
    if t in ("todos", "degradado"):
        exp_degradado(a.reps)
    if t in ("todos", "gantt"):
        exp_gantt()


if __name__ == "__main__":
    main()
```

3 graficas.py

Genera las figuras a partir de los resultados.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
graficas.py - Genera las figuras (PNG) a partir de resultados/*.json.

    python graficas.py

Salida: resultados/figuras/*.png  (fondo oscuro, listas para Beamer)
"""
import json
import os

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

AQUI = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(AQUI, "resultados")
FIG = os.path.join(RES, "figuras")

# Tema: "oscuro" (por defecto) o "claro" (variable de entorno TEMA=claro)
TEMA = os.environ.get("TEMA", "oscuro")
if TEMA == "formal":
    FIG = os.path.join(RES, "figuras_formal")
    SUPERFICIE, TINTA, TINTA2, REJILLA = "#ffffff", "#111111", "#555555", "#dddddd"
    COLOR = {"dinamico": "#1f3a68", "estatico": "#9b2335", "guiado": "#2e7d7a",
             "estatico_ciclico": "#b8860b", "estatico_ponderado": "#7a6a9c"}
elif TEMA == "claro":
    FIG = os.path.join(RES, "figuras_claro")
    SUPERFICIE, TINTA, TINTA2, REJILLA = "#f6f3ec", "#23201c", "#6f685c", "#ddd7cb"
    COLOR = {"dinamico": "#2f6b5a", "estatico": "#9c3d54", "guiado": "#4c63a8",
             "estatico_ciclico": "#c9962b", "estatico_ponderado": "#d9743a"}
else:
    SUPERFICIE, TINTA, TINTA2, REJILLA = "#1a1a19", "#f0efec", "#c3c2b7", "#3a3a38"
    COLOR = {"dinamico": "#3987e5", "estatico": "#d95926", "guiado": "#199e70",
             "estatico_ciclico": "#c98500", "estatico_ponderado": "#d55181"}
os.makedirs(FIG, exist_ok=True)
NOMBRE = {"estatico": "Estático (bloques)", "estatico_ciclico": "Estático cíclico",
          "estatico_ponderado": "Estático ponderado", "guiado": "Guiado"}

# Tipografía opcional (Kalam) si está en la carpeta fuentes/ del proyecto
FUENTE = {}
_kal = os.path.join(AQUI, "fuentes", "Kalam-Regular.ttf")
if TEMA == "formal":
    FUENTE = {"font.family": "serif", "font.serif": ["Times New Roman", "Liberation Serif", "TeX Gyre Termes", "DejaVu Serif"]}
if TEMA == "claro" and os.path.exists(_kal):
    from matplotlib import font_manager
    font_manager.fontManager.addfont(_kal)
    font_manager.fontManager.addfont(os.path.join(AQUI, "fuentes", "Kalam-Bold.ttf"))
    FUENTE = {"font.family": "Kalam"}

# En el tema claro las figuras se hacen más chicas (texto relativamente más grande)
ESCALA_CLARO = 0.78 if TEMA in ("claro", "formal") else 1.0
_subplots = plt.subplots


def _subplots_escalado(*a, **kw):
    if "figsize" in kw:
        kw["figsize"] = tuple(x * ESCALA_CLARO for x in kw["figsize"])
    return _subplots(*a, **kw)


plt.subplots = _subplots_escalado

plt.rcParams.update({
    "figure.facecolor": SUPERFICIE, "axes.facecolor": SUPERFICIE,
    "savefig.facecolor": SUPERFICIE, "text.color": TINTA,
    "axes.labelcolor": TINTA2, "xtick.color": TINTA2, "ytick.color": TINTA2,
    "axes.edgecolor": REJILLA, "grid.color": REJILLA,
    "font.size": {"claro": 9.5, "formal": 10.5}.get(TEMA, 11),
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 12.5, "axes.titleweight": "bold", "axes.titlecolor": TINTA,
    **FUENTE,
})


def cargar(nombre):
    with open(os.path.join(RES, nombre + ".json"), encoding="utf-8") as f:
        return json.load(f)


def nombre(d):
    if d["estrategia"] == "dinamico":
        return f"Dinámico (paquete={d['chunk']})"
    return NOMBRE[d["estrategia"]]


```

```python
def guardar(fig, archivo):
    fig.savefig(os.path.join(FIG, archivo), dpi=200, bbox_inches="tight")
    plt.close(fig)
    print("  figura:", archivo)


# ------------------------------------------------------------------ 1. Gantt
def gantt():
    corridas = [cargar("gantt_estatico"), cargar("gantt_dinamico")]
    fig, ejes = plt.subplots(2, 1, figsize=(8.2, 4.3), sharex=True)
    xmax = max(c["makespan"] for c in corridas) * 1.06
    for ax, c in zip(ejes, corridas):
        col = COLOR[c["estrategia"]]
        trab = c["trabajadores_detalle"]
        for fila, t in enumerate(trab):
            y = len(trab) - 1 - fila
            fin_ultimo = 0
            for ini, fin, n in t["intervalos"]:
                ax.barh(y, fin - ini, left=ini, height=0.62, color=col,
                        edgecolor=SUPERFICIE, linewidth=1.6)
                fin_ultimo = max(fin_ultimo, fin)
            # tiempo ocioso: desde que termina hasta el final de la corrida
            if c["makespan"] - fin_ultimo > 0.02:
                ax.barh(y, c["makespan"] - fin_ultimo, left=fin_ultimo, height=0.62,
                        facecolor="none", edgecolor=TINTA2, hatch="////",
                        linewidth=0, alpha=0.55)
        ax.axvline(c["t_ideal"], color=TINTA2, linestyle=(0, (4, 3)), linewidth=1.2)
        ax.set_yticks(range(len(trab)))
        ax.set_yticklabels([f"Nodo {t['rank']}  (v={t['velocidad']:g})"
                            for t in reversed(trab)])
        ax.set_title(f"{nombre(c)}  -  {c['makespan']:.2f} s", loc="left")
        ax.set_xlim(0, xmax)
        ax.grid(axis="x", linewidth=0.6)
        ax.set_axisbelow(True)
    ejes[-1].set_xlabel("Tiempo (s)")
    ejes[1].legend(handles=[Patch(facecolor=COLOR["dinamico"], label="trabajando"),
                            Patch(facecolor="none", edgecolor=TINTA2, hatch="////",
                                  label="ocioso (esperando)"),
                            Line2D([0], [0], color=TINTA2, linestyle=(0, (4, 3)),
                                   label="tiempo ideal")],
                   loc="center right", frameon=False, labelcolor=TINTA2, fontsize=9.5)
    fig.tight_layout()
    guardar(fig, "gantt.png")


# ------------------------------------------------- 2. makespan por estrategia
def barras_makespan():
    escenarios = [("homogeneo", "Nodos iguales  (1, 1, 1, 1)"),
                  ("heterogeneo", "Nodos distintos  (1, 1, 0.5, 0.25)")]
    fig, ejes = plt.subplots(1, 2, figsize=(9, 3.4), sharex=True, sharey=False)
    xmax = 0
    for ax, (arch, titulo) in zip(ejes, escenarios):
        datos = cargar(arch)
        ys = list(range(len(datos)))[::-1]
        for y, d in zip(ys, datos):
            ax.barh(y, d["makespan_mediana"], height=0.62, color=COLOR[d["estrategia"]])
            ax.text(d["makespan_mediana"] + 0.04, y, f"{d['makespan_mediana']:.2f} s",
                    va="center", color=TINTA, fontsize=9.5)
            xmax = max(xmax, d["makespan_mediana"])
        ax.set_yticks(ys)
        ax.set_yticklabels([nombre(d) for d in datos], fontsize=9.5)
        ax.set_title(titulo, loc="left", fontsize=14)
        ax.grid(axis="x", linewidth=0.6)
        ax.set_axisbelow(True)
        ax.set_xlabel("Tiempo total (s)  -  menos es mejor")
    for ax in ejes:
        ax.set_xlim(0, xmax * 1.18)
    fig.tight_layout()
    guardar(fig, "makespan.png")


# ---------------------------------------- 3. ocupado vs ocioso por nodo
def ocio_por_nodo():
    corridas = [cargar("gantt_estatico"), cargar("gantt_dinamico")]
    fig, ejes = plt.subplots(1, 2, figsize=(9, 3.4), sharey=True)
    for ax, c in zip(ejes, corridas):
        col = COLOR[c["estrategia"]]
        trab = c["trabajadores_detalle"]
        xs = range(len(trab))
        ocupado = [t["ocupado"] for t in trab]
        ocioso = [t["ocioso"] for t in trab]
        ax.bar(xs, ocupado, width=0.62, color=col, edgecolor=SUPERFICIE, linewidth=2)
        ax.bar(xs, ocioso, width=0.62, bottom=ocupado, facecolor="none",
               edgecolor=TINTA2, hatch="////", linewidth=0, alpha=0.6)
        for x, t in zip(xs, trab):
            ax.text(x, c["makespan"] + 0.05, f"{100 * t['utilizacion']:.0f} %",
                    ha="center", color=TINTA, fontsize=9.5)
        ax.set_xticks(list(xs))
        ax.set_xticklabels([f"Nodo {t['rank']}\nv={t['velocidad']:g}" for t in trab])
        ax.set_title(f"{nombre(c)}", loc="left")
        ax.set_ylim(0, max(x["makespan"] for x in corridas) * 1.18)
        ax.grid(axis="y", linewidth=0.6)
        ax.set_axisbelow(True)
```

```
    ejes[0].set_ylabel("Tiempo (s)")
    ejes[1].legend(handles=[Patch(facecolor=COLOR["dinamico"], label="ocupado"),
                            Patch(facecolor="none", edgecolor=TINTA2, hatch="////",
                                  label="ocioso")],
                   loc="upper right", frameon=False, labelcolor=TINTA2, fontsize=9.5)
    fig.tight_layout()
    guardar(fig, "ocio_nodos.png")


# ------------------------------------------------ 4. tamaño de paquete
def chunk():
    datos = cargar("chunk")
    din = [d for d in datos if d["estrategia"] == "dinamico"]
    guiado = next(d for d in datos if d["estrategia"] == "guiado")
    pond = next(d for d in datos if d["estrategia"] == "estatico_ponderado")
    fig, (a1, a2) = plt.subplots(2, 1, figsize=(8.2, 4.4), sharex=True,
                                 gridspec_kw={"height_ratios": [3, 1.6]})
    xs = list(range(len(din)))
    ys = [d["makespan_mediana"] for d in din]
    a1.plot(xs, ys, color=COLOR["dinamico"], linewidth=2.4, marker="o",
            markersize=9, markeredgecolor=SUPERFICIE, markeredgewidth=2)
    a1.axhline(guiado["makespan_mediana"], color=COLOR["guiado"], linewidth=1.8,
               linestyle=(0, (5, 3)))
    a1.axhline(pond["makespan_mediana"], color=COLOR["estatico_ponderado"],
               linewidth=1.8, linestyle=(0, (5, 3)))
    a1.text(len(xs) - 1.05, guiado["makespan_mediana"] + 0.04, "Guiado",
            color=COLOR["guiado"], ha="right", fontsize=9.5)
    a1.text(len(xs) - 1.05, pond["makespan_mediana"] - 0.13, "Estático ponderado",
            color=COLOR["estatico_ponderado"], ha="right", fontsize=9.5)
    a1.set_ylabel("Tiempo total (s)")
    a1.set_title("Paquete chico = muchos mensajes . paquete grande = nodos lentos frenan",
                 loc="left", fontsize=10.5)
    a1.grid(axis="y", linewidth=0.6)
    a1.set_ylim(0.7, max(ys) * 1.1)
    a2.bar(xs, [d["paquetes"] for d in din], width=0.6, color=COLOR["dinamico"],
           alpha=0.75)
    for x, d in zip(xs, din):
        a2.text(x, d["paquetes"] * 1.15, str(d["paquetes"]), ha="center",
                color=TINTA2, fontsize=9.5)
    a2.set_yscale("log")
    a2.set_ylim(1, 900)
    a2.set_ylabel("Paquetes\n(log)")
    a2.set_xticks(xs)
    a2.set_xticklabels([str(d["chunk"]) for d in din])
    a2.set_xlabel("Tareas por paquete (chunk)")
    a2.grid(axis="y", linewidth=0.6)
    fig.tight_layout()
    guardar(fig, "chunk.png")


# ------------------------------------------------ 5. escalabilidad
def escalabilidad():
    datos = cargar("escalabilidad")
    base = [1, 1, 0.5, 0.25]
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ws = [1, 2, 4, 8]
    ideal = []
    for w in ws:
        v = [base[i % 4] for i in range(w)]
        ideal.append(sum(v) / max(v))
    ax.plot(ws, ideal, color=TINTA2, linewidth=1.6, linestyle=(0, (5, 3)), marker="o",
            markersize=6)
    ax.text(8.12, ideal[-1], "ideal", color=TINTA2, va="center", fontsize=9.5)
    for est, chunk_ in (("dinamico", 4), ("guiado", 1), ("estatico", 1)):
        serie = [d for d in datos if d["estrategia"] == est and d["chunk"] == chunk_]
        serie.sort(key=lambda d: d["trabajadores"])
        sp = [d["t_1_nodo"] / d["makespan_mediana"] for d in serie]
        ax.plot(ws, sp, color=COLOR[est], linewidth=2.6, marker="o", markersize=9,
                markeredgecolor=SUPERFICIE, markeredgewidth=2)
        etiqueta = {"dinamico": "Dinámico", "guiado": "Guiado", "estatico": "Estático"}[est]
        dy = {"dinamico": 0.28, "guiado": -0.42, "estatico": 0}[est]
        ax.text(8.12, sp[-1] + dy, etiqueta, color=COLOR[est], va="center", fontsize=10.5)
    ax.set_xticks(ws)
    ax.set_xlim(0.7, 9.6)
    ax.set_xlabel("Número de trabajadores (nodos)")
    ax.set_ylabel("Speedup vs. el nodo más rápido solo")
    ax.grid(axis="y", linewidth=0.6)
    ax.set_axisbelow(True)
    fig.tight_layout()
    guardar(fig, "escalabilidad.png")


# ------------------------------------------------ 6. nodo degradado
def degradado():
    datos = cargar("degradado")
    fig, ax = plt.subplots(figsize=(7.4, 2.9))
    ys = list(range(len(datos)))[::-1]
    for y, d in zip(ys, datos):
        ax.barh(y, d["makespan_mediana"], height=0.6, color=COLOR[d["estrategia"]])
        ax.text(d["makespan_mediana"] + 0.03, y, f"{d['makespan_mediana']:.2f} s",
                va="center", fontsize=10.5)
    ax.set_yticks(ys)
    ax.set_yticklabels([nombre(d) for d in datos])
```

```
    ax.set_xlim(0, max(d["makespan_mediana"] for d in datos) * 1.18)
    ax.set_xlabel("Tiempo total (s)  -  menos es mejor")
    ax.grid(axis="x", linewidth=0.6)
    ax.set_axisbelow(True)
    fig.tight_layout()
    guardar(fig, "degradado.png")


# ------------------------------------------------ 7. tareas por nodo
def tareas_por_nodo():
    est = cargar("gantt_estatico")["trabajadores_detalle"]
    din = cargar("gantt_dinamico")["trabajadores_detalle"]
    total = sum(t["tareas"] for t in din)
    suma_v = sum(t["velocidad"] for t in din)
    fig, ax = plt.subplots(figsize=(6.4, 3.3))
    ancho = 0.34
    for i, (e, d) in enumerate(zip(est, din)):
        ax.bar(i - ancho / 2 - 0.02, e["tareas"], ancho, color=COLOR["estatico"])
        ax.bar(i + ancho / 2 + 0.02, d["tareas"], ancho, color=COLOR["dinamico"])
        ideal = total * d["velocidad"] / suma_v
        ax.plot([i - 0.45, i + 0.45], [ideal, ideal], color=TINTA2, linewidth=1.8,
                linestyle=(0, (4, 3)))
        ax.text(i + ancho / 2 + 0.02, d["tareas"] + 2, str(d["tareas"]), ha="center",
                color=TINTA, fontsize=9.5)
        ax.text(i - ancho / 2 - 0.02, e["tareas"] + 2, str(e["tareas"]), ha="center",
                color=TINTA, fontsize=9.5)
    ax.set_xticks(range(len(din)))
    ax.set_xticklabels([f"Nodo {t['rank']}\nv={t['velocidad']:g}" for t in din])
    ax.set_ylabel("Tareas que ejecutó")
    ax.set_ylim(0, max(t["tareas"] for t in din) * 1.2)
    ax.grid(axis="y", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.legend(handles=[Patch(facecolor=COLOR["estatico"], label="Estático (60 c/u)"),
                       Patch(facecolor=COLOR["dinamico"], label="Dinámico"),
                       Line2D([0], [0], color=TINTA2, linestyle=(0, (4, 3)),
                              label="proporcional a v")],
              loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=3, frameon=False,
              labelcolor=TINTA2, fontsize=9.5)
    fig.tight_layout()
    guardar(fig, "tareas_nodo.png")


# ------------------------------------------------ 8. tamaño de paquetes guiado
def paquetes_guiado(n=240, w=4, factor=2.0, minimo=1):
    import math
    tam, restantes = [], n
    while restantes > 0:        # misma fórmula que tam_paquete() de balanceo.py
        t = min(restantes, max(minimo, math.ceil(restantes / (factor * w))))
        tam.append(t)
        restantes -= t
    fig, ax = plt.subplots(figsize=(7, 3))
    ax.bar(range(1, len(tam) + 1), tam, color=COLOR["guiado"], width=0.7)
    ax.axhline(4, color=COLOR["dinamico"], linewidth=2, linestyle=(0, (5, 3)))
    ax.text(len(tam), 5.2, "Dinámico fijo: siempre 4", color=COLOR["dinamico"],
            ha="right", fontsize=9.5)
    ax.text(1.4, tam[0] - 1.5, f"{tam[0]} tareas", color=TINTA, fontsize=9.5)
    ax.set_xlabel("Número de paquete entregado")
    ax.set_ylabel("Tareas en el paquete")
    ax.grid(axis="y", linewidth=0.6)
    ax.set_axisbelow(True)
    fig.tight_layout()
    guardar(fig, "paquetes_guiado.png")


# ------------------------------------------------ 9. tiempos en dos escenarios
def tiempos_dos_escenarios():
    escenarios = [("heterogeneo", "Nodos distintos  (1, 1, 0.5, 0.25)"),
                  ("degradado", "Un nodo se degrada a mitad")]
    fig, ejes = plt.subplots(1, 2, figsize=(9, 3.1), sharex=True)
    xmax = 0
    for ax, (arch, titulo) in zip(ejes, escenarios):
        datos = cargar(arch)
        ys = list(range(len(datos)))[::-1]
        for y, d in zip(ys, datos):
            ax.barh(y, d["makespan_mediana"], height=0.62, color=COLOR[d["estrategia"]])
            ax.text(d["makespan_mediana"] + 0.04, y, f"{d['makespan_mediana']:.2f} s",
                    va="center", color=TINTA, fontsize=10)
            xmax = max(xmax, d["makespan_mediana"])
        ax.set_yticks(ys)
        ax.set_yticklabels([nombre(d) for d in datos], fontsize=9.5)
        ax.set_title(titulo, loc="left", fontsize=11.5)
        ax.grid(axis="x", linewidth=0.6)
        ax.set_axisbelow(True)
        ax.set_xlabel("Tiempo total (s)  -  menos es mejor")
    for ax in ejes:
        ax.set_xlim(0, xmax * 1.2)
    fig.tight_layout()
    guardar(fig, "tiempos.png")


if __name__ == "__main__":
    gantt()
    barras_makespan()
```

```
    ocio_por_nodo()
    chunk()
    escalabilidad()
    degradado()
    tareas_por_nodo()
    paquetes_guiado()
    tiempos_dos_escenarios()
```

4 tabla_latex.py

Convierte los resultados en macros y tablas de LaTeX.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
tabla_latex.py - Convierte resultados/*.json en macros y tablas de LaTeX
para que los números de la presentación siempre coincidan con tus corridas.

    python tabla_latex.py      ->  presentacion/numeros.tex y tabla_resumen.tex
"""
import json
import os

AQUI = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(AQUI, "resultados")
OUT = os.path.join(AQUI, "presentacion")
os.makedirs(OUT, exist_ok=True)


def cargar(n):
    with open(os.path.join(RES, n + ".json"), encoding="utf-8") as f:
        return json.load(f)


def buscar(lista, est, chunk=None):
    for d in lista:
        if d["estrategia"] == est and (chunk is None or d["chunk"] == chunk):
            return d
    raise KeyError((est, chunk))


het, hom = cargar("heterogeneo"), cargar("homogeneo")
deg, chk, esc = cargar("degradado"), cargar("chunk"), cargar("escalabilidad")
t = lambda d: f"{d['makespan_mediana']:.2f}"

est, cic = buscar(het, "estatico"), buscar(het, "estatico_ciclico")
pon, d1 = buscar(het, "estatico_ponderado"), buscar(het, "dinamico", 1)
d4, gui = buscar(het, "dinamico", 4), buscar(het, "guiado")
din_chk = [d for d in chk if d["estrategia"] == "dinamico"]
mejor = min(din_chk, key=lambda d: d["makespan_mediana"])
peor = max(din_chk, key=lambda d: d["makespan_mediana"])
e8 = [d for d in esc if d["trabajadores"] == 8]

macros = {
    "tEstHet": t(est), "tCicHet": t(cic), "tPondHet": t(pon), "tDinUnoHet": t(d1),
    "tDinHet": t(d4), "tGuiHet": t(gui),
    "tEstHom": t(buscar(hom, "estatico")), "tDinHom": t(buscar(hom, "dinamico", 4)),
    "tIdealHet": f"{est['t_ideal']:.2f}", "tUnNodoHet": f"{est['t_1_nodo']:.2f}",
    "tEstDeg": t(buscar(deg, "estatico")), "tCicDeg": t(buscar(deg, "estatico_ciclico")),
    "tDinDeg": t(buscar(deg, "dinamico", 4)), "tGuiDeg": t(buscar(deg, "guiado")),
    "spEstHet": f"{est['t_1_nodo'] / est['makespan_mediana']:.2f}",
    "spDinHet": f"{d4['t_1_nodo'] / d4['makespan_mediana']:.2f}",
    "efDinHet": f"{100 * d4['t_ideal'] / d4['makespan_mediana']:.0f}",
    "efEstHet": f"{100 * est['t_ideal'] / est['makespan_mediana']:.0f}",
    "utEstHet": f"{100 * est['utilizacion_media']:.0f}",
    "utDinHet": f"{100 * d4['utilizacion_media']:.0f}",
    "mejoraHet": f"{est['makespan_mediana'] / d4['makespan_mediana']:.1f}",
    "mejoraDeg": f"{buscar(deg, 'estatico')['makespan_mediana'] / buscar(deg, 'dinamico', 4)['makespan_mediana']:.1f}",
    "chunkMejor": str(mejor["chunk"]), "tChunkMejor": t(mejor),
    "chunkPeor": str(peor["chunk"]), "tChunkPeor": t(peor),
    "spDinOcho": f"{buscar(e8, 'dinamico', 4)['t_1_nodo'] / buscar(e8, 'dinamico', 4)['makespan_mediana']:.1f}",
    "spEstOcho": f"{buscar(e8, 'estatico')['t_1_nodo'] / buscar(e8, 'estatico')['makespan_mediana']:.1f}",
}
with open(os.path.join(OUT, "numeros.tex"), "w", encoding="utf-8") as f:
    f.write("% Generado por tabla_latex.py - no editar a mano\n")
    for k, v in macros.items():
        f.write(f"\\newcommand{{\\{k}}}{{{v}}}\n")

nombres = {"estatico": "Estático (bloques)", "estatico_ciclico": "Estático cíclico",
           "estatico_ponderado": "Estático ponderado", "dinamico": "Dinámico",
           "guiado": "Guiado"}
filas = []
for d in het:
    n = nombres[d["estrategia"]]
    if d["estrategia"] == "dinamico":
        n += f" (paq.\\,{d['chunk']})"
    filas.append(f"{n} & {d['makespan_mediana']:.2f} & "
                 f"{d['t_1_nodo'] / d['makespan_mediana']:.2f}$\\times$ & "
                 f"{100 * d['t_ideal'] / d['makespan_mediana']:.0f}\\,\\% & "
                 f"{100 * d['utilizacion_media']:.0f}\\,\\% & "
                 f"{d['ocioso_total']:.2f} & {d['mensajes']} \\\\")
tabla = ("\\begin{tabular}{@{}lrrrrrr@{}}\n\\toprule\n"
         "Estrategia & Tiempo (s) & Speedup & Efic. & Util. & Ocio (s) & Msj. \\\\\n"
         "\\midrule\n" + "\n".join(filas) + "\n\\bottomrule\n\\end{tabular}\n")
with open(os.path.join(OUT, "tabla_resumen.tex"), "w", encoding="utf-8") as f:
    f.write(tabla)
print("listo: presentacion/numeros.tex y presentacion/tabla_resumen.tex")
```

5 cluster_diagnostico.py

Diagnóstico del clúster de laptops: IP, latencia al maestro, velocidad relativa y GPU.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""cluster_diagnostico.py - Diagnostico de un cluster de laptops en red local (mpi4py).

Para cada nodo (proceso MPI) mide lo que necesita el balanceo de carga:
  * nombre de host y direccion IP          -> comprobar que MPI llego a cada laptop
  * latencia de ida y vuelta (RTT) al maestro -> decide el tamano del paquete
  * velocidad relativa v (1.0 = el mas rapido) -> alimenta --velocidades
  * si hay GPU NVIDIA (nvidia-smi)         -> informativo (CPU/GPU)

Uso (una laptop):
    mpiexec -n 3 python cluster_diagnostico.py
Uso (cluster; hosts.txt con una IP por linea, ver cluster/hosts.txt.ejemplo):
    mpiexec -f cluster/hosts.txt -n 4 python cluster_diagnostico.py
Windows (MS-MPI):
    mpiexec -hosts 4 192.168.1.10 1 192.168.1.11 1 192.168.1.12 1 192.168.1.13 1 python cluster_diagnostico.py
"""
import argparse
import math
import shutil
import socket
import statistics
import time

from mpi4py import MPI


def ip_local():
    """IP de la interfaz que usaria para salir a la red (no envia paquetes)."""
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(("10.255.255.255", 1))
        return s.getsockname()[0]
    except OSError:
        return socket.gethostbyname(socket.gethostname())
    finally:
        s.close()


def trabajo_cpu(n):
    """Carga fija: cuenta primos hasta n (mismo trabajo en todos los nodos)."""
    cuenta = 0
    for k in range(2, n):
        for d in range(2, int(math.isqrt(k)) + 1):
            if k % d == 0:
                break
        else:
            cuenta += 1
    return cuenta


def main():
    ap = argparse.ArgumentParser(description=__doc__.split("\n")[0])
    ap.add_argument("--pings", type=int, default=200, help="ida y vuelta por nodo")
    ap.add_argument("--n-primos", type=int, default=60000, help="tamano del benchmark")
    ap.add_argument("--costo-medio-ms", type=float, default=10.0,
                    help="costo medio de una tarea (para sugerir el paquete)")
    args = ap.parse_args()

    comm = MPI.COMM_WORLD
    rank, size = comm.Get_rank(), comm.Get_size()

    # 1) latencia: rank 0 hace ping-pong con cada trabajador
    rtt = {}
    for w in range(1, size):
        comm.Barrier()
        if rank == 0:
            muestras = []
            for _ in range(args.pings):
                t0 = time.perf_counter()
                comm.send(b"x", dest=w, tag=1)
                comm.recv(source=w, tag=2)
                muestras.append((time.perf_counter() - t0) * 1000)
            rtt[w] = statistics.median(muestras)
        elif rank == w:
            for _ in range(args.pings):
                comm.recv(source=0, tag=1)
                comm.send(b"x", dest=0, tag=2)

    # 2) velocidad: todos hacen el mismo trabajo y se compara el tiempo
    trabajo_cpu(2000)                       # calentamiento
    comm.Barrier()
    t0 = time.perf_counter()
    trabajo_cpu(args.n_primos)
    t_cpu = time.perf_counter() - t0

    info = (rank, socket.gethostname(), ip_local(), t_cpu,
            shutil.which("nvidia-smi") is not None)
    todos = comm.gather(info, root=0)
```

```
90
91      if rank == 0:
92          t_min = min(t[3] for t in todos if t[0] != 0) if size > 1 else todos[0][3]
93          print(f"{'rank':>4}  {'host':<16} {'IP':<15} {'RTT ms':>7} {'t cpu s':>8} {'v':>5}  GPU")
94          vs = []
95          for r, host, ip, t, gpu in todos:
96              v = t_min / t
97              etiqueta = "maestro" if r == 0 else ""
98              lat = "-" if r == 0 else f"{rtt[r]:.3f}"
99              print(f"{r:>4}  {host:<16} {ip:<15} {lat:>7} {t:>8.3f} {min(v, 1.0):>5.2f}  "
100                   f"{'si' if gpu else 'no'} {etiqueta}")
101             if r != 0:
102                 vs.append(min(v, 1.0))
103         if vs:
104             L = max(rtt.values()) / 2                  # latencia de un sentido (ms)
105             k = max(1, math.ceil(18 * L * max(vs) / args.costo_medio_ms))
106             print("\nVelocidades para --velocidades :", ",".join(f"{v:.2f}" for v in vs))
107             print(f"Latencia de un sentido L = {L:.3f} ms")
108             print(f"Paquete sugerido (sobrecosto <= 10 %): k >= 18*L*v/c = {k} tareas "
109                   f"(c = {args.costo_medio_ms:g} ms)")
110
111
112 if __name__ == "__main__":
113     main()
```

6 servidor_demo.py

Servidor local de la demo en vivo: ejecuta `mpiexec` con parámetros validados y devuelve los intervalos reales.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""servidor_demo.py - Demo en vivo: una página web que ejecuta balanceo.py con MPI de verdad.

    python servidor_demo.py            (abre http://127.0.0.1:8765 en el navegador)

La página manda los parámetros de cada paso; este servidor lanza
    mpiexec -n N python balanceo.py ...        (sin shell, parámetros validados)
y devuelve la salida de la terminal y los intervalos reales de cada nodo para animarlos.
Solo escucha en 127.0.0.1 (tu propia laptop). Ctrl+C para detenerlo.
"""
import json
import os
import re
import shutil
import subprocess
import sys
import tempfile
import threading
import webbrowser
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

RAIZ = os.path.dirname(os.path.abspath(__file__))
PUERTO = int(os.environ.get("PUERTO_DEMO", "8765"))
ESTRATEGIAS = {"estatico", "estatico_ciclico", "estatico_ponderado", "dinamico", "guiado"}
MPIEXEC = shutil.which("mpiexec") or shutil.which("mpiexec.exe") or shutil.which("mpirun")
CANDADO = threading.Lock()          # una ejecución a la vez (para medir bien los tiempos)
RE_DEGRADAR = re.compile(r"^\d{1,2}:\d{1,3}(\.\d+)?:\d{1,2}(\.\d+)?$")


def numero(valor, minimo, maximo, nombre):
    x = float(valor)
    if not (minimo <= x <= maximo):
        raise ValueError(f"{nombre} fuera de rango ({minimo} a {maximo})")
    return x


def correr(cmd, timeout):
    env = dict(os.environ, PYTHONIOENCODING="utf-8")
    p = subprocess.run(cmd, cwd=RAIZ, capture_output=True, timeout=timeout, env=env)
    return p.returncode, p.stdout.decode("utf-8", "replace"), p.stderr.decode("utf-8", "replace")


def con_mpi(args_python, n, timeout):
    """Ejecuta mpiexec -n N python <args>. Reintenta con --oversubscribe (Open MPI)."""
    if not MPIEXEC:
        raise RuntimeError("No encuentro mpiexec en el PATH. Instala MS-MPI (Windows) o MPICH/Open MPI.")
    base = [MPIEXEC, "-n", str(n), sys.executable] + args_python
    codigo, salida, error = correr(base, timeout)
    if codigo != 0 and "slots" in (error + salida).lower():
        base = [MPIEXEC, "--oversubscribe", "-n", str(n), sys.executable] + args_python
        codigo, salida, error = correr(base, timeout)
    return codigo, salida, error, base


def ejecutar_balanceo(p):
    estrategia = str(p.get("estrategia", "dinamico"))
    if estrategia not in ESTRATEGIAS:
        raise ValueError("estrategia no válida")
    vel = [numero(v, 0.05, 4, "velocidad") for v in str(p.get("velocidades", "1,1,0.5,0.25")).split(",")]
    if not (1 <= len(vel) <= 8):
        raise ValueError("de 1 a 8 trabajadores")
    chunk = int(numero(p.get("chunk", 4), 1, 1000, "chunk"))
    lat = numero(p.get("latencia_ms", 0), 0, 200, "latencia")
    tareas = int(numero(p.get("tareas", 240), 20, 2000, "tareas"))
    degradar = str(p.get("degradar", "") or "")
    if degradar and not RE_DEGRADAR.match(degradar):
        raise ValueError("degradar debe ser rank:segundos:velocidad")
    vel_txt = ",".join(f"{v:g}" for v in vel)

    with tempfile.TemporaryDirectory() as tmp:
        salida_json = os.path.join(tmp, "resultado.json")
        args = ["balanceo.py", "--estrategia", estrategia, "--chunk", str(chunk),
                "--velocidades", vel_txt, "--latencia-ms", f"{lat:g}", "--tareas", str(tareas),
                "--guardar-intervalos", "--salida", salida_json]
        if degradar:
            args += ["--degradar", degradar]
        codigo, salida, error, cmd = con_mpi(args, len(vel) + 1, 120)
        if codigo != 0 or not os.path.exists(salida_json):
            raise RuntimeError((error or salida or "mpiexec terminó con error").strip()[-1500:])
        with open(salida_json, encoding="utf-8") as f:
            resultado = json.load(f)
    return {"ok": True, "salida": salida, "resultado": resultado}


def ejecutar_diagnostico(_p):
    codigo, salida, error, cmd = con_mpi(["cluster_diagnostico.py", "--pings", "100"], 3, 120)
    if codigo != 0:
        raise RuntimeError((error or salida).strip()[-1500:])
```

```
    return {"ok": True, "salida": salida}


class Manejador(BaseHTTPRequestHandler):
    def _enviar(self, codigo, cuerpo, tipo="application/json; charset=utf-8"):
        datos = cuerpo if isinstance(cuerpo, bytes) else json.dumps(cuerpo).encode("utf-8")
        self.send_response(codigo)
        self.send_header("Content-Type", tipo)
        self.send_header("Content-Length", str(len(datos)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(datos)

    def _local(self):
        host = (self.headers.get("Host") or "").split(":")[0]
        return host in ("127.0.0.1", "localhost")

    def do_GET(self):
        if not self._local():
            return self._enviar(403, {"ok": False, "error": "solo localhost"})
        if self.path in ("/", "/index.html"):
            with open(os.path.join(RAIZ, "demo", "index.html"), "rb") as f:
                return self._enviar(200, f.read(), "text/html; charset=utf-8")
        if self.path == "/favicon.ico":
            return self._enviar(204, b"", "image/x-icon")
        if self.path == "/api/estado":
            return self._enviar(200, {"ok": True, "mpiexec": MPIEXEC, "python": sys.executable,
                                      "version": sys.version.split()[0]})
        self._enviar(404, {"ok": False, "error": "no existe"})

    def do_POST(self):
        if not self._local() or self.path not in ("/api/ejecutar", "/api/diagnostico"):
            return self._enviar(403, {"ok": False, "error": "no permitido"})
        try:
            largo = min(int(self.headers.get("Content-Length") or 0), 10000)
            datos = json.loads(self.rfile.read(largo) or b"{}")
            with CANDADO:
                if self.path == "/api/diagnostico":
                    r = ejecutar_diagnostico(datos)
                else:
                    r = ejecutar_balanceo(datos)
            self._enviar(200, r)
        except subprocess.TimeoutExpired:
            self._enviar(200, {"ok": False, "error": "Tiempo agotado: mpiexec tardó más de 2 minutos."})
        except Exception as e:                                  # noqa: BLE001
            self._enviar(200, {"ok": False, "error": str(e)})

    def log_message(self, fmt, *a):
        sys.stderr.write("  " + (fmt % a) + "\n")


def main():
    srv = ThreadingHTTPServer(("127.0.0.1", PUERTO), Manejador)
    url = f"http://127.0.0.1:{PUERTO}"
    print(f"Demo en vivo en {url}   (Ctrl+C para salir)")
    print("mpiexec:", MPIEXEC or "NO ENCONTRADO")
    try:
        webbrowser.open(url)
    except Exception:                                           # noqa: BLE001
        pass
    try:
        srv.serve_forever()
    except KeyboardInterrupt:
        print("\nServidor detenido.")


if __name__ == "__main__":
    main()
```

7 cluster/hosts.txt.ejemplo

```
# Ejemplo de archivo de hosts (MPICH / Hydra). Una laptop por linea: IP:procesos
# Copia este archivo a cluster/hosts.txt y cambia las IP por las de tus laptops
# (en Windows: ipconfig, en Linux/Mac: ip a / ifconfig).
# La primera linea es la laptop donde se lanza el maestro (rank 0).
192.168.1.10:1
192.168.1.11:1
192.168.1.12:1
192.168.1.13:1
```